The Linux Command Line
A Complete Introduction (2nd Edition)

by William Shotts (No Starch Press)

Comprehensive Book Summary — with Worked Examples

Chapter-by-chapter overview with commands/tools, key takeaways, and a practical example for each

Prepared as a reading companion — not a substitute for the full text.

About This Summary

This document summarizes all 36 chapters of *The Linux Command Line, 2nd Edition* by William Shotts. This revised version adds a **runnable example** under every key concept, so each bullet point is paired with a small, concrete command-line illustration rather than just a description.

Each chapter section includes: a synopsis, key concepts (each followed by a shaded example box showing real command syntax/output), and a box listing every **command/tool** covered. Commands appearing for the **first time** in the book are flagged in orange.

Book Structure (4 Parts, 36 Chapters)

- **Part I — Learning the Shell** (Ch. 1–10): terminal basics, navigation, exploring the system, file manipulation, commands, redirection, expansion/quoting, keyboard tricks, permissions, processes.
- **Part II — Configuration and the Environment** (Ch. 11–13): environment variables, the vi editor, and customizing the shell prompt.
- **Part III — Common Tasks and Essential Tools** (Ch. 14–23): package management, storage media, networking, searching, archiving/backup, regular expressions, text processing, output formatting, printing, and compiling programs.
- **Part IV — Writing Shell Scripts** (Ch. 24–36): from a first script through variables, functions, all flow-control constructs (if/case/while/for), input handling, troubleshooting, strings/numbers/arrays, and advanced ("exotica") shell features.

Part I — Learning the Shell

Chapter 1: What Is the Shell?

Introduces the shell as a text-based command interface to Linux, accessed through a terminal emulator. Covers launching a terminal, making first keystrokes (command history, cursor movement), trying a few simple commands (date, cal, df), and properly ending a terminal session.

Key concepts & examples

• The shell is a program that reads typed commands and asks the kernel to execute them.

```
$ date
Sat Oct 24 10:15:02 UTC 2026
```

• Most Linux distributions default to bash (Bourne Again SHell).

```
$ echo $SHELL
/bin/bash
```

• A terminal session can be ended with the exit command or by closing the terminal window.

```
$ exit
```

Commands / tools covered in this chapter

`date`	`cal`	`df`	`exit`	

★ Newly introduced in this chapter: date, cal, df

Chapter 2: Navigation

Covers the Linux filesystem tree structure, the concept of the current working directory, and how to move around it. Introduces absolute vs. relative pathnames and handy shortcuts (~, ., .., -) for faster navigation.

Key concepts & examples

• The filesystem is a single unified tree rooted at /, unlike drive-letter systems.

```
$ ls /
bin boot dev etc home usr var ...
```

• pwd shows the current directory; cd changes it; absolute paths start at /, relative paths start from the current location.

```
$ pwd
/home/alice
$ cd /etc
$ cd Documents
```

• Shortcuts: ~ (home), . (current dir), .. (parent dir), - (previous dir) speed up navigation.

```
$ cd ~
$ cd ..
$ cd -
```

Commands / tools covered in this chapter

`pwd`	`cd`			

★ Newly introduced in this chapter: pwd, cd

Chapter 3: Exploring the System

Deepens directory listing with ls options and long-format output (permissions, owner, size, timestamps), determining file types with file, and viewing file contents with less. Takes a guided tour of key system directories, then explains symbolic and hard links.

Key concepts & examples

• ls -l's long format reveals file type, permissions, link count, owner, group, size, and modification time in one view.

```
$ ls -l /etc/hosts
-rw-r--r-- 1 root root 220 Jan 10 09:00 /etc/hosts
```

• less is the standard pager for viewing (not editing) file contents, supporting search.

```
$ less /etc/passwd
(press / then type a word to search, q to quit)
```

• Symbolic links are pointers to a path; hard links are additional names for the same inode.

```
$ ln -s /etc/hosts myhosts
$ ls -l myhosts
lrwxrwxrwx 1 alice alice 10 ... myhosts -> /etc/hosts
```

Commands / tools covered in this chapter

`ls`	`file`	`less`		

★ Newly introduced in this chapter: ls, file, less

Chapter 4: Manipulating Files and Directories

Covers wildcards (globbing patterns) for matching multiple filenames, then the core file-management commands: mkdir, cp, mv, rm, and ln. A hands-on "playground" exercise has the reader practice creating, copying, moving, linking, and removing files/directories.

Key concepts & examples

• Wildcards (*, ?, [...]) let a single command operate on many matching files at once.

```
$ ls *.txt
$ rm file?.log
```

• cp/mv/rm all support recursive (-r) and interactive/force options for safety and bulk operations.

```
$ cp -r project/ project_backup/
$ rm -i old_file.txt
```

• ln creates hard links by default, or symbolic links with -s.

```
$ ln original.txt hardlink.txt
$ ln -s original.txt softlink.txt
```

Commands / tools covered in this chapter

`mkdir`	`cp`	`mv`	`rm`	`ln`

★ Newly introduced in this chapter: mkdir, cp, mv, rm, ln

Chapter 5: Working with Commands

Explains what commands actually are (executables, builtins, functions, aliases) and how to identify them with type and which. Covers getting documentation via help, --help, man, apropos, whatis, and info, plus README files, and creating custom commands with alias.

Key concepts & examples

• type reveals how the shell will interpret a word: alias, builtin, file, or function.

```
$ type cd
cd is a shell builtin
$ type ls
ls is /usr/bin/ls
```

• man is the canonical reference; apropos/whatis search man page summaries by keyword.

```
$ man ls
$ apropos partition
```

• alias lets users define their own shorthand commands (e.g., alias ll='ls -l').

```
$ alias ll='ls -l'
$ ll
```

Commands / tools covered in this chapter

type	which	help	man	apropos
whatis	info	alias		

★ Newly introduced in this chapter: type, which, man, apropos, whatis, info, alias

---

Chapter 6: Redirection

Covers the standard input/output/error streams and redirecting them to/from files (>, >>, 2>, &>), plus cat for concatenating/viewing files. Introduces pipelines (|) for chaining commands, and a set of classic filter utilities: uniq, wc, grep, head/tail, and tee.

Key concepts & examples

• Every process has three default streams: stdin (0), stdout (1), stderr (2), all independently redirectable.

```
$ ls /nonexistent 2> errors.txt
$ ls > out.txt 2>&1
```

• Pipelines connect one command's stdout directly to the next command's stdin, enabling powerful command composition.

```
$ ls -l | grep '^d' | wc -l
```

• grep (pattern matching) and the head/tail/wc/uniq filter family are foundational tools reused constantly throughout the rest of the book.

```
$ sort names.txt | uniq -c
$ tail -n 20 /var/log/syslog
```

Commands / tools covered in this chapter

cat	uniq	wc	grep	head
tail	tee			

★ Newly introduced in this chapter: cat, uniq, wc, grep, head, tail, tee

---

Chapter 7: Seeing the World as the Shell Sees It

Explains shell expansion mechanisms the shell performs before executing a command: pathname expansion (wildcards), tilde expansion, arithmetic expansion, brace expansion, parameter expansion, and command substitution. Also covers quoting (double/single) and escape sequences to control or prevent expansion.

Key concepts & examples

• Understanding expansion order is essential to predicting what a command actually receives as arguments.

```
$ echo file{1..3}.txt
file1.txt file2.txt file3.txt
```

• Command substitution ($(command)) embeds a command's output directly into another command line.

```
$ echo "Today is $(date +%A)"
Today is Saturday
```

• Double quotes suppress most expansion except parameter/command/arithmetic expansion; single quotes suppress all expansion.

```
$ echo "$HOME has $(ls | wc -l) files"
$ echo 'Literal $HOME is not expanded'
```

Commands / tools covered in this chapter

echo (expansion demonstration)				

★ Newly introduced in this chapter: echo

Chapter 8: Advanced Keyboard Tricks

Covers efficient command-line editing (cursor movement, text modification, cut/paste "killing and yanking"), tab completion, and using command history — including searching history and history expansion (e.g., !!, !n) for rapidly reissuing past commands.

Key concepts & examples

• Readline-based editing shortcuts (Ctrl-A/E/U/K/Y, etc.) dramatically speed up command-line use.

```
Ctrl-A (jump to start of line)
Ctrl-E (jump to end of line)
Ctrl-K (cut to end of line)
```

• Tab completion reduces typing and typos for filenames, commands, and variables.

```
$ cd Doc<TAB>
-> cd Documents/
```

• History expansion (!!, !$, !string) lets users reuse or reference prior commands concisely.

```
$ sudo !!
$ cp report.txt !$
```

Commands / tools covered in this chapter

history				

★ Newly introduced in this chapter: history

Chapter 9: Permissions

Explains Linux's owner/group/other permission model and the read/write/execute bits, then chmod (symbolic and octal modes) for changing them, umask for default permissions, and special permissions (setuid, setgid, sticky bit). Covers changing user/group identity with su and sudo, and ownership with chown/chgrp.

Key concepts & examples

• Every file/directory has three permission sets (owner, group, other) each with read/write/execute.

```
$ ls -l script.sh
-rwxr-xr-- 1 alice staff 512 ... script.sh
```

• chmod accepts both symbolic (u+x) and numeric/octal (755) modes.

```
$ chmod u+x script.sh
$ chmod 755 script.sh
```

• sudo (execute as another user, typically root) is the modern, audited alternative to su for privileged commands.

```
$ sudo apt update
$ sudo chown root:root /etc/myconfig
```

Commands / tools covered in this chapter

chmod	umask	su	sudo	chown
chgrp	passwd			

★ Newly introduced in this chapter: chmod, umask, su, sudo, chown, chgrp, passwd

---

Chapter 10: Processes

Explains how Linux processes work (parent/child relationships, PIDs), viewing them statically with ps and dynamically with top, and controlling them: interrupting, backgrounding/foregrounding, and stopping. Covers Unix signals and sending them with kill/killall, plus safely shutting down the system.

Key concepts & examples

• Every running program is a process with a unique PID, viewable via ps or top.

```
$ ps aux | grep firefox
$ top
```

• Ctrl-C sends SIGINT; Ctrl-Z suspends a process (SIGTSTP); jobs/bg/fg manage background/foreground state.

```
$ sleep 100
^Z
[1]+ Stopped sleep 100
$ bg %1
```

• kill sends signals (not just "kill" the process) — e.g., SIGTERM for graceful termination, SIGKILL to force it.

```
$ kill 4821
$ kill -9 4821
$ killall firefox
```

Commands / tools covered in this chapter

ps	top	jobs	bg	fg
kill	killall	shutdown		

★ Newly introduced in this chapter: ps, top, jobs, bg, fg, kill, killall, shutdown

---

Part II — Configuration and the Environment

Chapter 11: The Environment

Explains what's stored in the shell environment (environment variables), how the environment is established at login via startup files (e.g., .bash_profile, .bashrc), and how to modify it safely using a text editor to edit startup files and activate changes.

Key concepts & examples

• Environment variables (e.g., PATH, HOME, PS1) configure shell and program behavior globally for the session.

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin
```

• Startup files run automatically at login/shell-start and are the standard place to persist environment customizations.

```
$ nano ~/.bashrc
(add: export EDITOR=vim)
```

• Changes to startup files require re-sourcing (source) or a new session to take effect.

```
$ source ~/.bashrc
```

Commands / tools covered in this chapter

`env / printenv`	`export`	`source`		

★ Newly introduced in this chapter: env/printenv, export, source

---

Chapter 12: A Gentle Introduction to vi

A practical tutorial on the vi text editor (ubiquitous on nearly every Unix-like system), covering starting/stopping, the three editing modes (normal, insert, command-line), cursor movement, basic editing (append, delete, cut/copy/paste, join lines), search-and-replace, and editing multiple files.

Key concepts & examples

• vi's modal editing (separate modes for navigation vs. typing) is the key conceptual hurdle for newcomers but enables very fast editing once learned.

```
$ vi notes.txt
(press i to enter Insert mode, Esc to return to Normal mode)
```

• Knowing basic vi is valuable because it is available by default on virtually every Linux/Unix system, unlike more full-featured editors.

```
:wq (save and quit)
:q! (quit without saving)
```

• Global search-and-replace (:%s/old/new/g) mirrors sed syntax covered later in the book.

```
:%s/foo/bar/g
```

Commands / tools covered in this chapter

`vi / vim`				

★ Newly introduced in this chapter: vi/vim

Chapter 13: Customizing the Prompt

Covers the anatomy of the bash prompt (PS1 variable and its escape sequences), building alternative prompt designs, adding color with ANSI escape codes, moving the cursor within the prompt, and saving the customized prompt permanently in a startup file.

Key concepts & examples

• PS1 controls the primary prompt string using special backslash-escaped sequences (\u, \h, \w, etc.).

```
$ PS1='[\u@\h \W]\$ '
[alice@laptop ~]$
```

• ANSI color codes can be embedded in PS1 to visually distinguish prompt elements.

```
PS1='\[\e[32m\]\u@\h\[\e[0m\]:\w\$ '
```

• Prompt customizations should be saved in .bashrc to persist across sessions.

```
$ echo "PS1='[\u@\h \W]\$ '" >> ~/.bashrc
```

Commands / tools covered in this chapter

`PS1 (prompt variable)`				

Part III — Common Tasks and Essential Tools

Chapter 14: Package Management

Explains how Linux packaging systems work (package files, repositories, dependencies) and the difference between high-level tools (e.g., apt, yum/dnf) and low-level tools (dpkg, rpm). Walks through common tasks: finding, installing, removing, and updating packages, listing installed packages, and finding which package owns a given file.

Key concepts & examples

• High-level tools (apt-get/apt, yum/dnf) resolve dependencies automatically; low-level tools (dpkg, rpm) operate on individual package files directly.

```
$ sudo apt install htop
$ sudo dpkg -i package.deb
```

• Repositories are curated collections of packages the package manager pulls from.

```
$ sudo apt update
$ apt search nginx
```

• Tools exist to trace a file back to the package that installed it, useful for troubleshooting.

```
$ dpkg -S /bin/ls
coreutils: /bin/ls
```

Commands / tools covered in this chapter

`apt-get / apt`	`dpkg`	`yum / dnf`	`rpm`	

★ Newly introduced in this chapter: apt-get/apt, dpkg, yum/dnf, rpm

---

Chapter 15: Storage Media

Covers mounting/unmounting storage devices, viewing mounted filesystems and determining device names, creating new filesystems with fdisk (partitioning) and mkfs, testing/repairing filesystems, and moving data directly to/from devices (creating and writing CD-ROM/ISO images).

Key concepts & examples

• mount/umount attach and detach filesystems to the directory tree at a chosen mount point.

```
$ sudo mount /dev/sdb1 /mnt/usb
$ sudo umount /mnt/usb
```

• fdisk manipulates partition tables; mkfs creates a new filesystem on a partition.

```
$ sudo fdisk -l
$ sudo mkfs -t ext4 /dev/sdb1
```

• ISO image creation/writing covers legacy optical media workflows still occasionally useful.

```
$ dd if=/dev/sr0 of=disk.iso bs=2048
```

Commands / tools covered in this chapter

`mount`	`umount`	`fdisk`	`mkfs`	`fsck`
`dd`				

★ Newly introduced in this chapter: mount, umount, fdisk, mkfs, fsck, dd

---

Chapter 16: Networking

Covers examining and monitoring a network (ping, traceroute, ip, netstat), transporting files over a network (ftp, the improved lftp, and wget for downloading), and secure remote communication with ssh, plus the related scp and sftp for secure file transfer.

Key concepts & examples

• ping/traceroute diagnose basic connectivity and routing; ip manages network interfaces; netstat reports connections/listening ports.

```
$ ping -c 4 example.com
$ ip addr show
```

• ssh provides encrypted remote shell access and is the backbone of secure remote Linux administration.

```
$ ssh alice@192.168.1.10
```

• scp and sftp layer secure file transfer on top of the SSH protocol.

```
$ scp report.pdf alice@server:/home/alice/
$ wget https://example.com/file.tar.gz
```

Commands / tools covered in this chapter

ping	traceroute	ip	netstat	ftp
lftp	wget	ssh	scp	sftp

★ Newly introduced in this chapter: ping, traceroute, ip, netstat, ftp, lftp, wget, ssh, scp, sftp

---

Chapter 17: Searching for Files

Covers locate for fast searches against a prebuilt filename database, and the far more powerful find command for complex searches using tests, operators, predefined actions, and user-defined actions. Introduces xargs for efficiently applying commands to find's output and discusses performance considerations.

Key concepts & examples

• locate is fast but relies on a periodically updated database; find searches the live filesystem and supports rich criteria.

```
$ locate httpd.conf
$ find /var/log -name '*.log'
```

• find's -exec action (or piping to xargs) lets search results feed directly into another command.

```
$ find . -name '*.tmp' -exec rm {} \;
```

• xargs batches arguments efficiently, avoiding "argument list too long" errors from naive loops.

```
$ find . -name '*.log' | xargs rm
```

Commands / tools covered in this chapter

locate	find	xargs		

★ Newly introduced in this chapter: locate, find, xargs

Chapter 18: Archiving and Backup

Covers compressing files with gzip and bzip2, archiving with tar (the classic combination with compression) and zip, and synchronizing files/directories with rsync, including using rsync over a network for efficient incremental backups.

Key concepts & examples

• gzip/bzip2 compress single files; tar bundles multiple files/directories into one archive, commonly compressed together (tar.gz, tar.bz2).

```
$ gzip bigfile.txt
$ tar -czvf backup.tar.gz Documents/
```

• zip is used mainly for cross-platform (Windows-compatible) archives.

```
$ zip -r archive.zip Photos/
```

• rsync synchronizes files efficiently by transferring only changed portions, ideal for backups and remote mirroring.

```
$ rsync -av Documents/ /mnt/backup/Documents/
```

Commands / tools covered in this chapter

`gzip`	`bzip2`	`tar`	`zip / unzip`	`rsync`

★ Newly introduced in this chapter: gzip, bzip2, tar, zip/unzip, rsync

---

Chapter 19: Regular Expressions

A thorough regex primer built around grep: metacharacters and literals, the "any character" wildcard, anchors, bracket expressions/character classes, POSIX character classes, POSIX basic vs. extended regex, alternation, and quantifiers (?, *, +, {}).

Key concepts & examples

• Regular expressions are a pattern-matching language used across many Unix text tools (grep, sed, find, vim, and more).

```
$ grep '^root' /etc/passwd
```

• POSIX Basic (BRE) vs. Extended (ERE) regex differ in how metacharacters must be escaped.

```
$ grep -E '(cat|dog)' animals.txt
```

• Quantifiers and bracket expressions are the core building blocks for matching real-world patterns like phone numbers or filenames.

```
$ grep -E '^[0-9]{3}-[0-9]{4}$' phonelist.txt
```

Commands / tools covered in this chapter

`grep (regex engine)`	`egrep / grep -E`			

★ Newly introduced in this chapter: egrep/grep -E

Chapter 20: Text Processing

A deep toolkit for manipulating text: revisiting cat/sort/uniq, slicing and dicing with cut, paste, and join, comparing text with comm, diff, and patch, and editing on the fly with tr (transliteration), sed (the stream editor, covered extensively), and aspell for spellchecking.

Key concepts & examples

• sort, cut, paste, and join form a powerful toolkit for restructuring tabular/line-based text data.

```
$ cut -d: -f1 /etc/passwd | sort
```

• diff/patch together form the classic workflow for distributing and applying source code changes.

```
$ diff old.c new.c > fix.patch
$ patch old.c < fix.patch
```

• sed enables non-interactive, scriptable text transformations across entire files or streams.

```
$ sed 's/foo/bar/g' input.txt > output.txt
```

Commands / tools covered in this chapter

`sort`	`cut`	`paste`	`join`	`comm`
`diff`	`patch`	`tr`	`sed`	`aspell`

★ Newly introduced in this chapter: sort, cut, paste, join, comm, diff, patch, tr, sed, aspell

---

Chapter 21: Formatting Output

Covers simple formatting tools: nl (number lines), fold (wrap lines), fmt (simple text formatting), pr (format for printing), and printf (precise formatted output). Introduces document formatting systems, specifically groff.

Key concepts & examples

• printf gives fine-grained control over output formatting, useful both interactively and heavily in shell scripts.

```
$ printf '%-10s %5d\n' Apples 42
```

• pr/fmt/fold prepare plain text for readable display or printing.

```
$ fold -w 60 longline.txt
```

• groff represents the classic Unix document-formatting/typesetting toolchain, still used for man pages themselves.

```
$ groff -man -Tascii mypage.1 | less
```

Commands / tools covered in this chapter

nl	fold	fmt	pr	printf
groff				

★ Newly introduced in this chapter: nl, fold, fmt, pr, printf, groff

---

Chapter 22: Printing

A brief history of printing technology, then practical Linux printing: preparing files with pr, sending print jobs with lpr (Berkeley-style) or lp (System V-style), monitoring/controlling print jobs with lpstat, lpq, and lprm/cancel.

Key concepts & examples

• Linux supports two historical printing command families (BSD-style and System V-style), both still present via CUPS.

```
$ lpr report.pdf
$ lp report.pdf
```

• pr and a2ps convert plain text into print-ready formatted output.

```
$ pr -h "Monthly Report" data.txt | lpr
```

• Monitoring tools let users check queue status and cancel stuck or unwanted print jobs.

```
$ lpq
$ lprm 42
```

Commands / tools covered in this chapter

pr	lpr	lp	a2ps	lpstat
lpq	lprm	cancel		

★ Newly introduced in this chapter: lpr, lp, a2ps, lpstat, lpq, lprm, cancel

---

Chapter 23: Compiling Programs

Explains what compiling is, then walks through compiling a real C program from source: obtaining source code, examining the source tree, building the program (typically via ./configure && make), and installing it (make install).

Key concepts & examples

• Compiling translates human-readable source code into machine-executable binaries.

```
$ gcc -o hello hello.c
$ ./hello
```

• The classic Unix build workflow (configure, make, make install) automates detecting the build environment and compiling/installing software from source.

```
$ ./configure
$ make
```

```
$ sudo make install
```

• Understanding this process is useful for installing software not available as a pre-built package.

```
$ tar xzf app-1.2.tar.gz
$ cd app-1.2
```

Commands / tools covered in this chapter

gcc	make	./configure		

★ Newly introduced in this chapter: gcc, make, ./configure

Part IV — Writing Shell Scripts

Chapter 24: Writing Your First Script

Defines shell scripts as text files of commands executed by the shell, and covers the mechanics of writing one: the shebang line, setting executable permissions, where to place scripts, and formatting conventions for readability.

Key concepts & examples

• The shebang (#!/bin/bash) tells the system which interpreter should run the script.

```
#!/bin/bash
echo "Hello, World!"
```

• chmod +x is required before a script can be executed directly.

```
$ chmod +x hello.sh
$ ./hello.sh
```

• Good formatting (indentation, line continuation with \) is emphasized from the very first script onward.

```
ls -l \
/etc
```

Commands / tools covered in this chapter

chmod +x	#!/bin/bash (shebang)			

Chapter 25: Starting a Project

Begins a running example project: a minimal document-generating script, then adding data, introducing variables and constants (including readonly), and here documents (<<EOF) for embedding multiline text directly in a script.

Key concepts & examples

• Variables store and reuse values throughout a script; constants (readonly) prevent accidental modification.

```
TITLE="System Report"
readonly TITLE
```

• Here documents provide a clean way to embed multiline literal text without many separate echo calls.

```
cat <<EOF
Report generated on $(date)
EOF
```

• The project built here is extended in nearly every subsequent chapter of Part IV.

```
#!/bin/bash
TITLE="System Report"
echo "$TITLE"
```

Commands / tools covered in this chapter

readonly	here documents (<<)			

★ Newly introduced in this chapter: readonly

Chapter 26: Top-Down Design

Introduces shell functions as a way to modularize scripts using top-down design principles, covers local variables (scoped to a function), and techniques to keep long-running scripts running robustly.

Key concepts & examples

• Shell functions let a script be decomposed into named, reusable blocks.

```
print_header () {
echo "=== $1 ==="
}
print_header "Report"
```

• local restricts a variable's scope to the function it's declared in.

```
count_files () {
local n=$(ls | wc -l)
echo $n
}
```

• Techniques like trapping signals help long scripts survive unexpected interruptions.

```
trap 'echo Interrupted; exit' SIGINT
```

Commands / tools covered in this chapter

function / local				

★ Newly introduced in this chapter: local

Chapter 27: Flow Control: Branching with if

Covers the if statement and exit status, the test command for evaluating file, string, and integer expressions, the modern [[ ]] syntax, (( )) for integers, and the && / || control operators.

Key concepts & examples

• Every command returns an exit status (0 = success) that if/while/until branch on.

```
grep -q root /etc/passwd
echo $?
```

• test/[ ]/[[ ]] evaluate conditions on files, strings, and integers.

```
if [ -f "$1" ]; then
echo "File exists"
fi
```

• && and || provide a compact way to chain conditional execution.

```
mkdir backup && cp *.txt backup/ || echo "Failed"
```

Commands / tools covered in this chapter

test	[ ]	[[ ]]	(( ))	

★ Newly introduced in this chapter: test/[ ], [[ ]], (( ))

Chapter 28: Reading Keyboard Input

Covers the read builtin for capturing user input, its options and the IFS variable, validating input, and building simple text-based menus.

Key concepts & examples

• read is the core builtin for interactive scripts that prompt the user for values.

```
read -p "Enter your name: " name
echo "Hello, $name"
```

• IFS determines how read splits input into multiple variables/fields.

```
IFS=: read user pass uid gid rest < /etc/passwd
```

• Combining read with case/if builds simple interactive menu-driven scripts.

```
read -p "Choice (1-3): " choice
case $choice in
1) echo one;;
esac
```

Commands / tools covered in this chapter

`read`				

★ Newly introduced in this chapter: read

Chapter 29: Flow Control: Looping with while/until

Covers the while loop and until loop, breaking out of loops early, and the common pattern of reading files line-by-line with a while read loop.

Key concepts & examples

• while and until are logical opposites, chosen based on which condition reads more naturally.

```
count=1
while [ $count -le 5 ]; do
echo $count
count=$((count+1))
done
```

• break and continue control loop execution from within.

```
for f in *; do
[ -d "$f" ] && continue
echo "$f"
done
```

• while read line is the idiomatic shell pattern for processing a file one line at a time.

```
while read -r line; do
echo "Line: $line"
done < file.txt
```

Commands / tools covered in this chapter

`while`	`until`	`break/continue`		

★ Newly introduced in this chapter: while, until

Chapter 30: Troubleshooting

Covers categories of scripting errors — syntactic and logical — then testing with deliberate test cases and debugging techniques: tracing execution and examining variable values.

Key concepts & examples

• Most beginner script bugs fall into predictable categories: missing quotes and unexpected word-splitting/globbing are especially common.

```
# Wrong: if [ $var = foo ] (fails if $var is empty)
# Right: if [ "$var" = "foo" ]
```

• bash's built-in tracing (set -x) shows each command as it executes, invaluable for debugging.

```
$ bash -x myscript.sh
```

• Defensive programming (checking inputs, quoting variables) prevents many bugs before they occur.

```
if [ -z "$1" ]; then
echo "Usage: $0 filename"
exit 1
fi
```

Commands / tools covered in this chapter

set -x (tracing)	bash -n (syntax check)			

★ Newly introduced in this chapter: set -x

Chapter 31: Flow Control: Branching with case

Covers the case command for pattern-based multi-way branching, its pattern syntax, and performing multiple actions for a single matched pattern.

Key concepts & examples

• case is ideal for matching a single value against multiple possible patterns.

```
case $1 in
start) echo starting;;
stop) echo stopping;;
*) echo "unknown";;
esac
```

• Patterns support wildcards and the | operator to match several alternatives in one clause.

```
case $file in
*.jpg|*.png) echo image;;
*.txt) echo text;;
esac
```

• case is generally more readable than an equivalent chain of if/elif/else statements.

```
case $(uname) in
Linux) echo "Linux system";;
Darwin) echo "macOS";;
esac
```

Commands / tools covered in this chapter

case				

★ Newly introduced in this chapter: case

Chapter 32: Positional Parameters

Covers accessing command-line arguments via positional parameters ($1, $2, ...), $#, shift, using positional parameters in functions, and $@ / $*.

Key concepts & examples

• Positional parameters let scripts accept command-line arguments just like built-in commands.

```
#!/bin/bash
echo "First arg: $1, total args: $#"
```

• shift repeatedly removes $1 and renumbers the rest, enabling loops over an arbitrary number of arguments.

```
while [ $# -gt 0 ]; do
echo "$1"
shift
done
```

• "$@" (quoted) is the standard idiom for safely passing all arguments through to another command.

```
log_all () { echo "$@" >> log.txt; }
log_all "$@"
```

Commands / tools covered in this chapter

shift	$@/$*/$#			

★ Newly introduced in this chapter: shift

Chapter 33: Flow Control: Looping with for

Covers the traditional shell for loop and the C-language-style for loop syntax for numeric iteration.

Key concepts & examples

• The traditional for word in list form iterates over any whitespace-separated list.

```
for name in Alice Bob Carol; do
echo "Hello, $name"
done
```

• The C-style for ((...)) form is preferred for classic counter-based numeric loops.

```
for ((i=0; i<5; i++)); do
echo "$i"
done
```

• Choosing the right for form depends on whether the loop iterates over data or a numeric range.

```
for f in *.txt; do
mv "$f" "${f%.txt}.bak"
done
```

Commands / tools covered in this chapter

for				

★ Newly introduced in this chapter: for

Chapter 34: Strings and Numbers

A deep dive into parameter expansion for string manipulation, arithmetic evaluation and expansion, number bases, bit operations, and bc for operations bash's integer arithmetic can't handle.

Key concepts & examples

• Parameter expansion supports powerful inline string operations without needing external tools.

```
file="report.txt"
echo "${file%.txt}" # report
echo "${file/report/summary}"
```

• Bash arithmetic is integer-only; bc is required for floating-point or very large calculations.

```
$ echo "scale=2; 10/3" | bc
3.33
```

• Mastering these expansions lets scripts avoid spawning external processes for simple tasks.

```
x=5
echo $((x * 2 + 1)) # 11
```

Commands / tools covered in this chapter

bc	$(( )) (arithmetic expansion)			

★ Newly introduced in this chapter: bc

Chapter 35: Arrays

Covers bash arrays: creating and assigning values, accessing elements and slices, array operations, and associative arrays.

Key concepts & examples

• Indexed arrays store ordered lists of values under a single variable name.

```
fruits=(apple banana cherry)
echo "${fruits[1]}" # banana
```

• Associative arrays (declare -A) provide key-value storage, similar to dictionaries in other languages.

```
declare -A colors
colors[apple]=red
echo "${colors[apple]}"
```

• Array operations (${#arr[@]}, ${!arr[@]}, arr+=(...)) are essential for scripts managing collections of data.

```
fruits+=(date)
echo "${#fruits[@]}" # 4
```

Commands / tools covered in this chapter

declare -a / declare -A (arrays)				

★ Newly introduced in this chapter: declare -a/-A

Chapter 36: Exotica

A grab-bag of advanced shell features: group commands and subshells (including process substitution), signal trapping with trap, asynchronous execution with wait, and named pipes (FIFOs).

Key concepts & examples

• trap lets a script catch signals (e.g., SIGINT/Ctrl-C) and run cleanup code before exiting.

```
trap 'rm -f /tmp/lockfile; exit' SIGINT SIGTERM
```

• wait synchronizes a script with background jobs it launched, useful for simple parallelism.

```
sleep 5 &
sleep 3 &
wait
echo "All background jobs done"
```

• Named pipes (mkfifo) allow two otherwise-unrelated processes to communicate via a filesystem-visible pipe.

```
$ mkfifo mypipe
$ echo hello > mypipe &
$ cat < mypipe
```

Commands / tools covered in this chapter

`trap`	`wait`	`mkfifo`		

★ Newly introduced in this chapter: trap, wait, mkfifo

Master Command Index (by first appearance)

This book teaches dozens of commands cumulatively, building toward full shell scripting. The table below groups every major command/utility by the chapter where it is first introduced, organized by functional category.

Category	Ch.	Commands
Navigation & file exploration	2–4	`pwd, cd, ls, file, less, mkdir, cp, mv, rm, ln`
Command help & identification	5	`type, which, help, man, apropos, whatis, info, alias`
Redirection & filters	6	`cat, uniq, wc, grep, head, tail, tee`
Shell interaction	7–8	`echo, history (expansion, completion, readline editing)`
Permissions & identity	9	`chmod, umask, su, sudo, chown, chgrp, passwd`
Processes	10	`ps, top, jobs, bg, fg, kill, killall, shutdown`
Environment & editors	11–13	`env/printenv, export, source, vi/vim, PS1 prompt customization`
Package management	14	`apt-get/apt, dpkg, yum/dnf, rpm`
Storage media	15	`mount, umount, fdisk, mkfs, fsck, dd`
Networking	16	`ping, traceroute, ip, netstat, ftp, lftp, wget, ssh, scp, sftp`
Searching	17	`locate, find, xargs`
Archiving & sync	18	`gzip, bzip2, tar, zip/unzip, rsync`
Regex & text processing	19–20	`grep/egrep, sort, cut, paste, join, comm, diff, patch, tr, sed, aspell`
Output formatting & printing	21–22	`nl, fold, fmt, pr, printf, groff, lpr, lp, a2ps, lpstat, lpq, lprm/cancel`
Compiling	23	`gcc, make, ./configure`
Shell scripting constructs	24–36	`test/[ ]/[[ ]], (( )), read, while/until, for, case, shift, bc, declare -a/`

Final Notes

The Linux Command Line is explicitly a learn-by-doing reference: nearly every chapter pairs explanation with hands-on exercises, and Part IV's running script project ties together nearly every concept from Parts I–III into real, working shell scripts. The worked examples added throughout this summary mirror that hands-on spirit — try them in a terminal to reinforce each concept.

The book's core value is cumulative: later chapters assume fluency with earlier commands (grep, sed, find, redirection, pipelines), and Part IV's scripting chapters combine nearly everything taught earlier into reusable automation. For a reader following the book start to finish, this summary's chapter order mirrors the intended learning path.